config.py — technical

src/moe_pruning/config.py · 405 lines · moe_pruning.config

Purpose

Single source of truth for settings. Produces an immutable `ExperimentConfig` dataclass from layered key/value sources and validates every field before any script does work.

Data and structure

- Input: `.env` path, optional profile paths, process environment, `--set` overrides.
- Output: `ExperimentConfig` (typed fields; `raw` keeps the merged string values) or `ConfigError` listing every problem.

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`class ConfigError`

lines 35-36 · bases ValueError

Raised when the configuration is missing or invalid.

`class ExperimentConfig`

lines 44-143 · dataclass

Frozen, validated settings of one run (one field per .env variable).

Field	Type	Default
`model_path`	str	
`hf_model_id`	str	
`hf_revision`	str | None	
`model_dtype`	str	
`model_plugins`	tuple[str, ...]	
`hf_cache_dir`	Path | None	
`experts_implementation`	str	
`device`	str	
`device_map`	str	
`max_memory`	str	
`calibration_dataset_path`	Path	
`evaluation_dataset_path`	Path	
`calibration_max_samples`	int	
`calibration_batch_size`	int	
`max_source_length`	int	
`max_target_length`	int	
`c4_dataset_id`	str	
`c4_config`	str	
`c4_split`	str	
`c4_domain`	str	
`c4_max_samples`	int	
`c4_max_scanned`	int	
`c4_min_text_length`	int	
`c4_max_text_length`	int	
`c4_min_keyword_matches`	int	

Field	Type	Default
`c4_support_keywords`	tuple[str, ...]	
`c4_url_keywords`	tuple[str, ...]	
`c4_allowed_domains`	tuple[str, ...]	
`pruning_method`	str	
`prune_experts_per_layer`	int	
`prune_layers`	str	
`prune_unobserved_policy`	str	
`replacement_method`	str	
`bias_initialization`	str	
`bias_training_objective`	str	
`bias_learning_rate`	float	
`bias_training_steps`	int	
`bias_batch_size`	int	
`bias_validation_fraction`	float	
`bias_eval_interval`	int	
`eval_max_samples`	int	
`eval_batch_size`	int	
`eval_noise_density`	float	
`eval_mean_span_length`	float	
`eval_seed`	int	
`eval_qualitative_samples`	int	
`seed`	int	
`experiment_name`	str	
`pruned_model_path`	Path	
`checkpoint_format`	str	
`experiment_dir`	Path	
`stats_dir`	Path	
`cache_dir`	Path	
`log_level`	str	'INFO'
`raw`	dict[str, str]	field(default_factory=dict, compare=False, repr=False)

`ExperimentConfig.run_dir`

```
def run_dir(self) -> Path
```

lines 115-117

Directory holding every artifact of the current experiment.

`ExperimentConfig.run_stats_dir`

```
def run_stats_dir(self) -> Path
```

lines 120-121

Statistics directory of this experiment.

`ExperimentConfig.run_cache_dir`

```
def run_cache_dir(self) -> Path
```

lines 124-125

Cache directory of this experiment.

`ExperimentConfig.model_source`

```
def model_source(self) -> str
```

lines 128-130

Local path if configured, otherwise the Hugging Face model id.

`ExperimentConfig.to_dict`

```
def to_dict(self) -> dict[str, Any]
```

lines 132-143

Convert to a plain dictionary for JSON reports.

1. Iterate dataclass fields except `raw`; convert `Path` to `str` and tuples to lists so the result is JSON-safe.

`class _Reader`

lines 149-217

1. Wraps the merged `dict[str, str]` and an `errors` list.
2. `_get(key, default)` treats missing keys and blank strings as 'unset' and returns the default.
3. Typed readers (`str`, `choice`, `int`, `float`, `path`) never raise: on bad input they append a readable message to `errors` and return a neutral value, so validation continues and all problems are reported together.
4. `choice` lower-cases the value before checking membership; `float` supports an exclusive minimum; `path` expands `~` and resolves relative paths against `PROJECT_ROOT` (two levels above `config.py`).

`_Reader.str`

```
def str(self, key: str, default: str | None=None, required: bool=True) -> str
```

lines 160-166

`_Reader.choice`

```
def choice(self, key: str, choices: tuple[str, ...], default: str | None=None) -> str
```

lines 168-172

`_Reader.int`

```
def int(self, key: str, default: int | None=None, minimum: int | None=None) -> int
```

lines 174-186

`_Reader.float`

```
def float(self, key: str, default: float | None=None, minimum: float | None=None, maximum: float |
None=None, exclusive_min: bool=False) -> float
```

lines 188-210

`_Reader.path`

```
def path(self, key: str, default: str | None=None) -> Path
```

lines 212-217

`_collect_values`

```
def _collect_values(env_file: Path | None, overrides: dict[str, str] | None, profiles: list[Path] |
None=None) -> dict[str, str]
```

lines 220-238 · raises ConfigError

1. Start with `dotenv_values(.env)` (only keys with a value).
2. Apply each profile file in order with `dotenv_values(profile)`; a missing profile raises `ConfigError` immediately.
3. Overlay real environment variables: every key already present plus every upper-case variable in `os.environ` (mirrors python-dotenv's `override=False`).
4. Apply `--set` overrides last.

load_config

```
def load_config(env_file: Path | str | None=DEFAULT_ENV_FILE, overrides: dict[str, str] | None=None,
profiles: list[Path | str] | None=None) -> ExperimentConfig
```

lines 241-363 · raises ConfigError · settings C4_MAX_TEXT_LENGTH, C4_MIN_TEXT_LENGTH, EXPERIMENT_NAME

Load and validate the configuration.

1. Resolve profile paths relative to the project root and call `_collect_values`.
2. Construct `ExperimentConfig(...)` field by field through the `_Reader` (defaults and minimum/maximum bounds are declared inline at each field).
3. Cross-field checks: `C4_MIN_TEXT_LENGTH < C4_MAX_TEXT_LENGTH`, non-empty `C4_SUPPORT_KEYWORDS`, `EXPERIMENT_NAME` without slashes.
4. If `MODEL_PATH` is non-empty it must be an existing directory containing `config.json` - otherwise an error (never a silent switch to `HF_MODEL_ID`).
5. Parse `DEVICE_MAP` and `MAX_MEMORY` with the placement parsers to surface format errors early.
6. If `errors` is non-empty raise one `ConfigError` with a bullet list; otherwise return the config.

add_config_arguments

```
def add_config_arguments(parser: argparse.ArgumentParser) -> argparse.ArgumentParser
```

lines 369-392

Add --env-file, --profile and --set to a script's argument parser.

config_from_args

```
def config_from_args(args: argparse.Namespace) -> ExperimentConfig
```

lines 395-405 · raises ConfigError

Build the configuration from parsed command-line arguments.

1. Split each `--set KEY=VALUE` on the first `=` (keys upper-cased); malformed items raise `ConfigError`.
2. Profiles come from `--profile` (repeatable) or, if none given, from the `MOE_PROFILE` environment variable.
3. Delegate to `load_config(args.env_file, overrides, profiles)`.

Module constants

Name	Value
PROJECT_ROOT	Path(__file__).resolve().parents[2]
DEFAULT_ENV_FILE	PROJECT_ROOT / '.env'
VALID_DTYPES	('float32', 'float16', 'bfloat16')
VALID_DEVICES	('auto', 'cuda', 'mps', 'cpu')
VALID_PRUNING_METHODS	('easyep',)
VALID_BIAS_INITIALIZATIONS	('gate_token_weighted_mean',)
VALID_TRAINING_OBJECTIVES	('layer_mse',)
VALID_UNOBSERVED_POLICIES	('exclude', 'allow')
VALID_REPLACEMENT_METHODS	('bias', 'drop', 'reroute')
VALID_CHECKPOINT_FORMATS	('full', 'delta')
VALID_EXPERTS_IMPLEMENTATIONS	('', 'eager', 'grouped_mm', 'batched_mm')

Errors raised

Function	Exception	Message (f-string fields in braces)
_collect_values	ConfigError	Profile {profile} does not exist
load_config	ConfigError	Invalid configuration: - ...
config_from_args	ConfigError	--set expects KEY=VALUE, got {item}